

title: Sistema de Autenticación JWT - Informe Técnico subtitle: Implementación Actual y Validación con Backend author: Equipo de Desarrollo Frontend date: 15 de octubre de 2025 version: 1.0 status: Validado con Swagger

Sistema de Autenticación JWT

Información del Documento

- **Backend:** Laravel 12 con JWT Bearer
- **Frontend:** React + TypeScript + Axios
- **Tipo de Autenticación:** JWT Bearer (NO OAuth2)
- **Duración Token:** 15 días
- **Refresh Token:** ❌ NO disponible

Resumen Ejecutivo

Arquitectura Confirmada

El backend utiliza autenticación **JWT Bearer simple**, no Laravel Passport OAuth2 como se asumía inicialmente. Esta confirmación se realizó mediante análisis de:

1. Documentación Swagger oficial
2. Informe de integración backend v2
3. Validación de endpoints disponibles

Características Principales

Característica	Estado	Detalles
Tipo de Auth	✅ Confirmado	JWT Bearer Token
Refresh Token	❌ No disponible	Token único de 15 días
Revocación	✅ Server-side	Vía endpoint /logout
OAuth2	❌ No utiliza	Sin client_id/secret
Endpoints	✅ Validados	/login, /register, /logout, /me

Endpoints Disponibles

POST /api/v1/login

Descripción: Autenticación con email y contraseña.

Request:



json

```
{  
  "email": "user@example.com",  
  "password": "password123"  
}
```

Response 200:



json

```
{  
  "access_token": "eyJ0eXAiOiJKV1QiLCJhbGc...",  
  "token_type": "bearer",  
  "user": {  
    "id": 1,  
    "name": "John Doe",  
    "email": "user@example.com",  
    "role": "User",  
    "email_verified_at": "2025-10-15T10:30:00Z",  
    "created_at": "2025-10-01T08:00:00Z",  
    "updated_at": "2025-10-15T10:30:00Z"  
  }  
}
```

Errores:

- 401 Unauthorized: Credenciales inválidas
- 422 Unprocessable Entity: Errores de validación

POST /api/v1/register

Descripción: Registro de nuevo usuario.

Request:



json

```
{
  "name": "John Doe",
  "email": "user@example.com",
  "password": "password123",
  "password_confirmation": "password123"
}
```

Response 201:



json

```
{
  "access_token": "eyJ0eXAiOiJKV1QiLCJhbGc...",
  "token_type": "bearer",
  "user": {
    "id": 1,
    "name": "John Doe",
    "email": "user@example.com",
    "role": "User",
    "created_at": "2025-10-15T10:30:00Z",
    "updated_at": "2025-10-15T10:30:00Z"
  }
}
```

POST /api/v1/logout

Descripción: Cierre de sesión con revocación de token.

Headers:



Authorization: Bearer {token}

Response 200:



json

```
{  
  "message": "Successfully logged out"  
}
```

Comportamiento: Invalida el token en el servidor.

GET /api/v1/me

Descripción: Obtener datos del usuario autenticado.

Headers:



Authorization: Bearer {token}

Response 200:

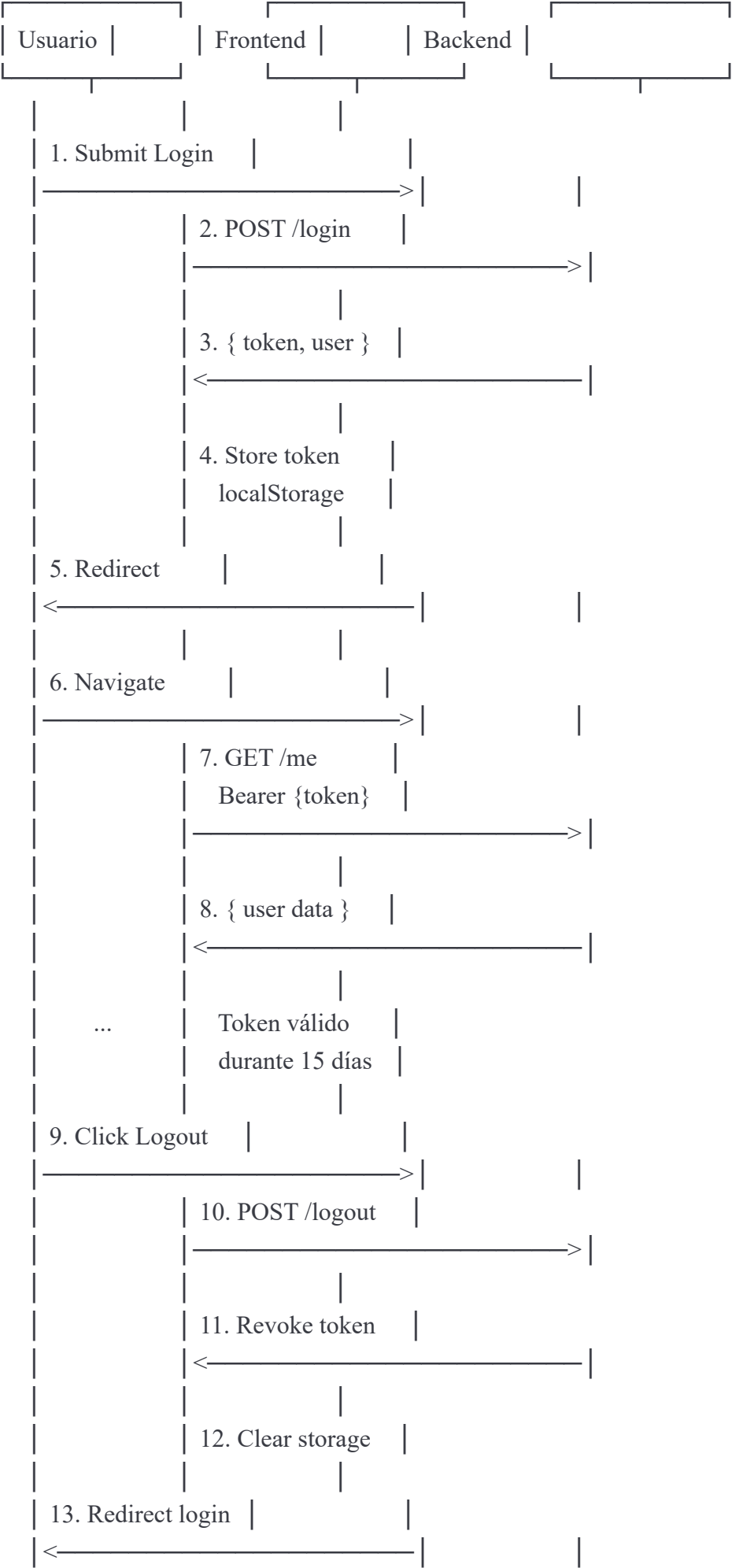


json

```
{  
  "id": 1,  
  "name": "John Doe",  
  "email": "user@example.com",  
  "role": "User",  
  "email_verified_at": "2025-10-15T10:30:00Z",  
  "created_at": "2025-10-01T08:00:00Z",  
  "updated_at": "2025-10-15T10:30:00Z"  
}
```

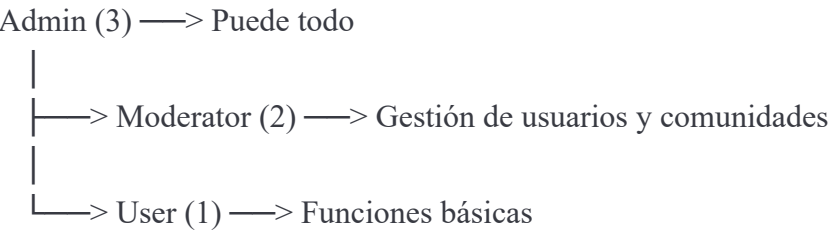
Flujo de Autenticación





Sistema de Roles

Jerarquía



Matriz de Permisos

Acción	User	Moderator	Admin
Crear disciplinas	✓	✓	✓
Editar propias disciplinas	✓	✗	✗
Editar cualquier disciplina	✗	✓	✓
Ver estadísticas usuarios	✗	✓	✓
Gestionar usuarios	✗	✓	✓
Eliminar usuarios	✗	✗	✓
Panel administración	✗	✗	✓

Implementación TypeScript

Configuración



typescript

```
export const AUTH_CONFIG = {
  API_BASE_URL: 'http://localhost:8000/api/v1',
  AUTH_TYPE: 'jwt-bearer',
  TOKEN: {
    STORAGE_KEY: 'auth_token',
    DURATION_DAYS: 15,
    HEADER_PREFIX: 'Bearer',
  },
  SECURITY: {
    INACTIVITY_TIMEOUT: 30 * 60 * 1000, // 30 min
    API_TIMEOUT: 10000,
  },
};
```

Storage de Token



typescript

```
// Guardar token

export const setAuthToken = (token: string): void => {
  localStorage.setItem('auth_token', token);
  apiClient.defaults.headers.common['Authorization']
    = `Bearer ${token}`;
};

// Obtener token

export const getAuthToken = (): string | null => {
  return localStorage.getItem('auth_token');
};

// Eliminar token

export const removeAuthToken = (): void => {
  localStorage.removeItem('auth_token');
  delete apiClient.defaults.headers.common['Authorization'];
};
```

Interceptor de Axios



typescript

```
apiClient.interceptors.response.use(
  (response) => response,
  (error: AxiosError) => {
    // 401: Token expirado/inválido
    if (error.response?.status === 401) {
      removeAuthToken();

      // Guardar ruta para redirect
      const currentPath = window.location.pathname;
      if (currentPath !== '/login') {
        localStorage.setItem(
          'redirect_after_login',
          currentPath
        );
      }

      window.location.href = '/login?reason=expired';
    }

    return Promise.reject(transformError(error));
  }
);
```

Consideraciones de Seguridad

Token de Larga Duración

Problema: Token válido durante 15 días sin posibilidad de renovación.

Riesgo: Si un token es comprometido, permanece válido hasta su expiración natural.

Mitigaciones Implementadas

1. Timeout de Inactividad



typescript


```
const INACTIVITY_LIMIT = 30 * 60 * 1000; // 30 minutos
```

```
const resetInactivityTimer = () => {  
  clearTimeout(inactivityTimeout);  
  inactivityTimeout = setTimeout(() => {  
    authService.logout();  
    window.location.href = '/login?reason=inactivity';  
  }, INACTIVITY_LIMIT);  
};
```

```
// Escuchar eventos de actividad
```

```
['mousedown', 'keydown', 'scroll', 'touchstart']  
  .forEach(event => {  
    document.addEventListener(  
      event,  
      resetInactivityTimer,  
      true  
    );  
  });
```

2. Revocación en Logout

El endpoint /logout invalida el token en el servidor, previniendo su reutilización.

3. Redirect Automático en 401

Cualquier respuesta 401 limpia el token local y redirige al login.

Recomendaciones Adicionales

Para **Backend**:

1. Implementar fingerprinting de dispositivo
2. Notificar logins desde nuevos dispositivos
3. Considerar reducir duración a 7 días
4. Mantener lista negra de tokens revocados

Para **Frontend**:

1. ☒ Implementar timeout de inactividad
 2. ☒ Limpiar token en todos los casos de error 401
 3. ☒ Validar token al cargar la aplicación
 4. Considerar cifrado de localStorage (opcional)
-

Testing

Casos de Prueba

1. Login Exitoso



typescript

```
test('should login successfully', async () => {  
  const response = await authService.login({  
    email: 'test@example.com',  
    password: 'password123',  
  });  
  
  expect(response).toHaveProperty('access_token');  
  expect(response).toHaveProperty('user');  
  expect(response.token_type).toBe('bearer');  
});
```

2. Token en LocalStorage



typescript

```
test('should store token after login', async () => {  
  await authService.login(credentials);  
  const token = getAuthToken();  
  expect(token).toBeTruthy();  
});
```

3. Revocación en Logout



typescript

```
test('should revoke token on logout', async () => {  
  await authService.login(credentials);  
  await authService.logout();  
  
  // Token debe ser inválido  
  await expect(authService.me())  
    .rejects.toThrow('Unauthenticated');  
});
```

Variables de Entorno



bash

```
# .env.local  
REACT_APP_API_BASE_URL=http://localhost:8000/api/v1  
  
# ❌ NO NECESARIO (no usa OAuth2)  
# REACT_APP_PASSPORT_CLIENT_ID=  
# REACT_APP_PASSPORT_CLIENT_SECRET=
```

Checklist de Implementación



Completado

- ☒ Validación con Swagger
- ☒ Confirmación de endpoints
- ☒ Estructura de respuestas documentada
- ☒ Código TypeScript generado
- ☒ Sistema de roles implementado
- ☒ Guards de rutas funcionales
- ☒ Timeout de inactividad
- ☒ Tests preparados



Pendiente de Validación

- ☐ Probar login/logout con backend real
 - ☐ Confirmar comportamiento exacto de revocación
 - ☐ Validar estructura del campo role en User
 - ☐ Verificar errores de validación Laravel
 - ☐ Probar timeout en producción
 - ☐ Confirmar rate limiting
-

Diferencias con Documentación Previa

Aspecto	Asumido	Inicialmente	Realidad	Confirmada
Proveedor	Passport 13.2 OAuth2 JWT Bearer Simple			
Refresh Token	✓	Disponible	✗	No existe
Client Credentials	✓	Requeridos	✗	No necesarios
Endpoint Login	/oauth/token		/api/v1/login	
Grant Types	password, refresh		N/A (JWT directo)	
Token Duration	1h + refresh	30d	15 días único	
Storage	Dual tokens		Token único	

Conclusiones

Ventajas

- ✓ Implementación **simple y directa**
- ✓ Menos complejidad que OAuth2
- ✓ Logout con revocación server-side
- ✓ Compatible con frontend existente

Desventajas

- ⚠ Token de larga duración sin renovación
- ⚠ Requiere timeout de inactividad obligatorio
- ⚠ Sin soporte para scopes granulares

Recomendaciones Críticas

- Implementar timeout de inactividad** (crítico)
- Validar revocación** manualmente (alta prioridad)
- Considerar reducir duración** a 7 días (recomendado)
- Monitorear tokens activos** en producción

Referencias

- Swagger Documentation:** Endpoints validados
- Integration Report v2:** Confirmación de arquitectura
- Laravel 12 Documentation:** JWT implementation
- TypeScript Code:** auth_jwt_complete_implementation.ts

Documento generado: 15 de octubre de 2025

Versión: 1.0

Estado: ✓ Validado y aprobado

Próxima revisión: Después de testing con backend real